

УКРАЇНСЬКИЙ КАТОЛИЦЬКИЙ УНІВЕРСИТЕТ

КУРС «ШТУЧНИЙ ІНТЕЛЕКТ» · СЕМЕСТРОВИЙ ПРОЄКТ · 2026

Квантування, дистиляція та прунінг великих мовних моделей для української мови

Який метод стискання дає найкращий компроміс між якістю, розміром, пам'яттю та швидкістю – для українськомовної моделі?

Команда:

Владислав Данилишин

Ія Магарита

Оксана Москв'як

Ментор:

Марта Сумик | Олександр Косован

Репозиторій:

<https://github.com/luftboud/small-LLM>

*Пропозиція проєкту, подана в межах курсу
«Штучний інтелект»*

Факультет прикладних наук



2026-09-21

1. Проблема та мотивація

Сучасні LLM через велику кількість параметрів потребують значного обсягу пам'яті та обчислювальних ресурсів, що суттєво ускладнює їх локальне використання та розгортання на пристроях з обмеженими ресурсами. Стиснення моделей за допомогою квантування, дистиляції та прунінгу може бути ефективним способом розв'язання цієї проблеми. Метою проєкту є порівняти ці методи, їхні різновиди та комбінації, щоб визначити, які з них забезпечують найкращий баланс між розміром моделі, споживанням пам'яті, швидкістю інференсу та якістю роботи, зокрема з українською мовою.

2. Огляд джерел та наявних рішень

Квантування переводить ваги й активації з високоточних числових форматів у формати з нижчою розрядністю. Основними підходами є Quantization-Aware Training (QAT), що враховує ефекти квантування під час навчання, і Post-Training Quantization (PTQ), що застосовується до вже навченої моделі. PTQ використовують частіше, оскільки він не потребує повторного навчання і тому є менш ресурсозатратним, однак на окремих завданнях це може призводити до погіршення якості моделі [1].

Прунінг видаляє незначні параметри або структурні компоненти LLM. Розрізняють три основні типи: Unstructured Pruning, що видаляє окремі ваги; Structured Pruning, що видаляє цілі компоненти моделі, наприклад нейрони, блоки або шари; і Semi-structured Pruning, що застосовує наперед визначені схеми розрідженості [2]. Особливо активно досліджують методи неструктурованого прунінгу, оскільки вони дозволяють досягати високої розрідженості з відносно невеликою втратою якості.

Дистиляція знань передбачає навчання компактнішої моделі-учня відтворювати поведінку більшої моделі-вчителя, зберігаючи її основні можливості за меншої кількості параметрів. Методи дистиляції LLM поділяють на Black-box KD, за якого модель-учень навчається лише на згенерованих відповідях моделі-вчителя, і White-box KD, за якого додатково використовуються розподіли ймовірностей або проміжні представлення моделі-вчителя [3]. Другий спосіб є дуже актуальним для моделей із відкритими вагами, оскільки дає змогу передавати моделі-учню більше внутрішньої інформації.

3. Дані

Тренування

- **Knowledge Distillation** потребує набору промптів, на яких велика модель дає цілі (текст або logits), а мала вчиться їх відтворювати – датасет від Lapa LLM [4].
- **Pruning** та **Quantization** потребують невеликих даних для калібрації. Pruning потребує дані, аби зрозуміти, які частини моделі найменш значущі, щоб їх прибрати. Quantization потребує малих даних для того, аби визначити межі для нового типу даних, щоб зменшити похибку та зберегти ефективність моделі. Для цих задач можна взяти меншу частину датасету згаданого вище.

Тестування

- Проєкт передбачає датасет для тестування точності мовної моделі до та після застосування оптимізаційних технік – MMLU [5]
- Датасет для оцінки сумаризації в порівнянні з еталоном – XLSum [6].
- Датасет для оцінки instruction-following сценаріїв – IFeval ukr [7].

4. Підхід і методи

Для ефективної оптимізації мовної моделі використовують різні техніки: Pruning, Quantization, Knowledge Distillation. Існує декілька інструментів, які можуть нам допомогти. Але перед тим потрібно визначитись, які саме методи будуть використані.

Метод квантизації. Є два методи квантизації: QAT (Quantization-Aware Training) та PTQ (Post-Training Quantization). Через своєрідну складність та обмеження в ресурсах використання QAT стає неможливим. Тому PTQ є тим методом, який ми можемо зробити.

Метод прунінгу. Можна робити прунінг на окремі ваги, тобто відслідковувати найменш впливові ваги та видаляти їх. А можна робити структурований прунінг, який видаляє цілі найменш впливові блоки. Ми будемо робити саме структурований прунінг.

NVIDIA NeMo. Цей інструмент допомагає зробити всі ці техніки завдяки своїм технологіям [8].

PyTorch. За допомогою нього можна зробити PTQ [9] та прунінг [10]

5. Оцінювання результатів

Baseline

План А:

lapa-llm/lapa-v0.1.3-instruct 12B, text-only, BF16 як еталон якості. [11] | *запуск на віддаленому instance АБО CPU offload (з вказанням цього моменту в звіті).*

План Б:

INSAIT-Institute/MamayLM-Gemma-3-4B-IT-v1.0, text-only, BF16 [12] | *локальний комп'ютер з 16 GB VRAM.*

Метрики і порівняння

Традиційні моделі ML оцінюють переважно за точністю або F1-мірою. Моделі категорії TinyML вартує оцінювати за сукупністю метрик [13]. Ми плануємо оцінювати моделі з допомогою метрик якості й ефективності.

Основний перелік **метрик якості** виглядає так:

- MMLU-ukr accuracy [5];
- IFEval strict accuracy [7];
- XL-Sum ROUGE-L [6, 14];
- Blind human evaluation.

А **метрики ефективності** це:

- розмір моделі, GB;
- peak RAM/VRAM, GB;
- швидкість генерації, tokens/s;
- час від надсилання промпта до першого токена, ms;
- energy usage на RUBIK Pi 3.

Підхід до порівняння

- однакові prompts, decoding, splits, batch size, input/output lengths і тд.;
- якість порівнювати по абсолютному score + retention;
- ресурси порівнювати з baseline;
- для benchmark scores виводити 95% confidence intervals;
- побудувати Pareto графіки + frontier.

6. План і ризики

1. **Підготовка:** Обрати baseline модель, провести аудит датасетів; Зафіксувати початкові показники базової моделі · Тижні 1-2
2. **Стискання:** Дослідити INT8/INT4-квантизацію, структурний прунінг і дистиляцію зі зменшенням

кількості шарів, проходження tutorials · Тижні 3-5

3. **Порівняння та мідтерм:** Мідтерм; Після проміжного захисту: застосувати стискання на baseline моделях; для кожної моделі отримати метрики; перевірити комбінації підходів та оцінити їх, визначити моделі з найкращим компромісом · Тижні 6-9
4. **Розгортання на edge-пристрої :** Конвертувати найкращу стиснену модель у сумісний формат і протестувати її запуск та продуктивність на RUBIK Pi 3; Отримати метрики · Тижні 9-11
5. **Аналіз:** Побудувати Pareto curves, провести human evaluation, error-analysis, забезпечити доказовість експерименту; Написати аналітичний звіт · Тижні 10-12
6. **Product phase:** Створити Comparison UI (як один промпт обробляється різними за розміром моделями); Упорядкувати репозиторій, написати вичерпну документацію; Створити постер та презентацію · Тижні 10-13

Ризики

1. 12B BF16 > 16 GB VRAM: CPU offload, що ускладнить порівняння швидкодії моделей за однакових умов.
2. Ресурс, який йде на тренування моделі, може обмежити дослідження різних підходів компресії.
3. Дистанційний доступ до RUBIK Pi 3.

7. Contribution statement

- **Владислав Данилишин** — відповідає за дослідження використання прунінгу, можливий instance
- **Ія Магарита** — відповідає за дослідження використання дистиляції, роботу з версіями моделей
- **Оксана Москв'як** — відповідає за дослідження використання квантизації, роботу з датасетами

Розподіл завдань між учасниками команди є гнучким і може змінюватися в процесі роботи.

Список джерел

- [1] Rui Huang. Edge-centric generative ai: A survey on efficient inference for large language models in resource-constrained environments. *Journal of Computer and Communications*, 14:238–253, 04 2026. doi:10.4236/jcc.2026.144012.
- [2] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics*, 12:1556–1577, 11 2024. arXiv:https://direct.mit.edu/tacL/article-pdf/doi/10.1162/tacL_a_00704/2482209/tacL_a_00704.pdf, doi:10.1162/tacL_a_00704.
- [3] Chuanpeng Yang, Yao Zhu, Wang Lu, Yidong Wang, Qian Chen, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. Survey on knowledge distillation for large language models: Methods, evaluation, and application. *ACM Trans. Intell. Syst. Technol.*, 16(6), November 2025. doi:10.1145/3699518.
- [4] lapa LLM. Ukrainian wiki instruction dialogs - dataset. <https://huggingface.co/datasets/lapa-llm/wiki-instruction-dialogs>. CC BY-SA 4.0 license.
- [5] INSAIT-Institute. mmlu ukr - dataset. https://huggingface.co/datasets/INSAIT-Institute/mmlu_ukr. Apache License 2.0 license.
- [6] csebuetnlp. Xl-sum: Large-scale multilingual abstractive summarization - dataset. <https://huggingface.co/datasets/csebuetnlp/xlsum>. CC BY-NC-SA 4.0 license.
- [7] INSAIT-Institute. Ifeval - dataset. https://huggingface.co/datasets/INSAIT-Institute/ifeval_ukr. Apache License 2.0 license.
- [8] NVIDIA. Nemo. <https://docs.nvidia.com/nemo-framework/user-guide/latest/model-optimization/quantization/quantization.html>. NeMa tools.
- [9] PyTorch. Practical quantization in pytorch. <https://pytorch.org/blog/quantization-in-practice/>.
- [10] PyTorch. Torch-pruning. <https://github.com/VainF/Torch-Pruning>.
- [11] Lapa LLM. Lapa llm v0.1.3. <https://huggingface.co/lapa-llm/lapa-v0.1.3-instruct>, 2026. Hugging Face model repository.
- [12] INSAIT-Institute. Mamaylm-gemma-3-4b-it-v1.0. <https://huggingface.co/INSAIT-Institute/MamayLM-Gemma-3-4B-IT-v1.0>, 2025. Hugging Face model repository.
- [13] Shriyank Somvanshi et al. From tiny machine learning to tiny deep learning: A survey. *J. ACM*, 37(4), 2025. URL: <https://arxiv.org/abs/2506.18927>.
- [14] PyTorch. Rouge-l. <https://docs.pytorch.org/ignite/generated/ignite.metrics.RougeL.html>. An evaluation metric.